

UART 实验指导书

一、UART 原理

1.1 要点

- **UART**：异步串行，常用 **TX / RX / GND**，波特率双方一致（本实验 **115200**），参数常写作 **8N1**（8 数据位、无校验、1 停止位）。
- **交叉接线**：本机 TX 接对方 RX，**共地**。
- 开发板 **USB 调试口**：经 CH340 等接到 PC 的 COM 口，`printf` 默认从此口输出。

1.2 行结束符

串口终端建议字符串末尾使用 `**\r\n**`。

1.3 三种使用方式

方式	典型 API	本指导书安排
轮询	<code>uapi_uart_read / uapi_uart_write</code>	第八节 ：照做即可
中断	<code>uapi_uart_register_rx_callback</code> 等	第九节 ：你写代码
DMA	<code>uapi_uart*_by_dma</code>	不要求；见 09-uapi_uart驱动接口与使用说明.md

二、创建 `uart_printf` 工程

在 OpenHarmony 源码**根目录**执行：

```
python3 applications/make_app.py -c uart_printf
```

若目录已存在，可用：

```
python3 applications/make_app.py -e uart_printf
```

说明：脚本会在 `applications/sample/wifi-iot/app/uart_printf/` 生成模板，并修改 `BUILD.gn`、`config.py`、`ohos.cmake`。目标名一般为 `**uart_printf_demo**`。

四、`uart_printf` 源码（请覆盖到工程目录）

路径：`applications/sample/wifi-iot/app/uart_printf/`

4.1 uart_printf.c

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */

#include <stdio.h>

#include "ohos_init.h"
#include "cmsis_os2.h"
#include "iot_watchdog.h"

#define UART_PRINTF_TASK_STACK_SIZE 0x1000
#define UART_PRINTF_TASK_PRIO      25

#define TICK_PER_SEC 100
#define MS_TO_TICKS(ms) (((ms) * TICK_PER_SEC) / 1000)

static void UartPrintfTask(void *arg)
{
    (void)arg;
    uint32_t seq = 1;

    while (1) {
        printf("Hello, Uart Printf! #%u\r\n", seq);
        seq++;
        IoTWatchDogKick();
        osDelay(MS_TO_TICKS(1000));
    }
}

static void UartPrintfEntry(void)
{
    osThreadAttr_t attr = {
        "UartPrintfTask", 0, NULL, 0, NULL, UART_PRINTF_TASK_STACK_SIZE,
        UART_PRINTF_TASK_PRIO, 0, 0
    };

    if (osThreadNew(UartPrintfTask, NULL, &attr) == NULL) {
        printf("[uart_printf] Failed to create UartPrintfTask!\r\n");
    }
}
```

```
APP_FEATURE_INIT(UartPrintfEntry);
```

4.2 BUILD.gn

```
# Copyright (c) 2026 SZTU Open Source Organization.
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

static_library("uart_printf_demo") {
    sources = [ "uart_printf.c" ]

    include_dirs = [
        "//commonlibrary/utls_lite/include",
        "//kernel/liteos_m/kal/cmsis",
        "//base/iothardware/peripheral/interfaces/inner_api",
    ]
}
```

五、编译

```
python3 applications/make_app.py -b
```

或全量：

```
python3 applications/make_app.py -f
```

六、烧录

固件路径示例：

```
out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/ws63-liteos-app_all.fwpkg
```

使用 BurnTool：选择 WS63、对应串口，烧录 ws63-liteos-app_all.fwpkg 后复位。详见 [03-环境配置和系统编译烧录文档.md](#)。

七、运行结果 (uart_printf)

串口助手：115200，8N1。复位后约每秒一行：

```
Hello, Uart Printf! #1
Hello, Uart Printf! #2
...
```

八、拓展实验一：轮询回显 (uart_uapi_poll)

8.1 目的

- 使用 `**uapi_uart_init**`、`**uapi_uart_read / uapi_uart_write**` 做回显。
- 理解：调试口 `printf` 与自己打开的 UART 总线可能不是同一路。

8.2 硬件

默认使用 `**UART_BUS_2`，`**S_MGPI08 (TX)`、`S_MGPI07 (RX)`，115200 8N1；第二路 USB 转 TTL 与排针 交叉接线、共地。勿默认占用 UART0（易与调试冲突）。无转接器时的折中做法与风险见完整版 第八节 说明。

8.3 步骤

1. 执行 `**python3 applications/make_app.py -c uart_uapi_poll**` (或 `**-e uart_uapi_poll**`)。
2. 用 8.4 的 `**uart_uapi_poll.c` 与 `BUILD.gn` 覆盖 `applications/sample/wifi-iot/app/uart_uapi_poll/` 下文件。
3. 在 `applications/sample/wifi-iot/app/BUILD.gn` 的 `**features**` 中启用 `**uart_uapi_poll:uart_uapi_poll_demo**` (做本实验时勿与第九节工程同时启用)。
4. 编译、烧录；调试串口应看到启动提示行；在 `MGPI08/MGPI07` 外接串口上发送字符应回显。

8.4 示例代码（请覆盖到工程目录）

路径： `applications/sample/wifi-iot/app/uart_uapi_poll/`

`uart_uapi_poll.c`

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
```

```

#include <stdio.h>

#include "errcode.h"
#include "uart.h"
#include "ohos_init.h"
#include "cmsis_os2.h"
#include "iot_watchdog.h"

/*
 * 使用 UART_BUS_2 + MGPI08(TX)/MGPI07(RX), 与 SDK BOARD_ASIC 下 UART2 复用一致 (见官方 IO 复用表:
 * GPIO_08=UART2_TXD, GPIO_07=UART2_RXD)。勿用 UART_BUS_0: 产品与 mconfig 中 CONFIG_DEBUG_UART
 * 等多为 UART0, 二次 uapi_uart_init 会与系统调试口争用导致异常/NMI。
 */
#define POLL_UART_BUS          UART_BUS_2
#define POLL_UART_TX_PIN      S_MGPI08
#define POLL_UART_RX_PIN      S_MGPI07
#define POLL_RX_BUF_SIZE      64
#define POLL_RW_TIMEOUT       0
#define POLL_TASK_STACK_SIZE  0x1000
#define POLL_TASK_PRIOR       25
#define TICK_PER_SEC          100
#define MS_TO_TICKS(ms)      (((ms) * TICK_PER_SEC) / 1000)

static uint8_t s_rx_buf[POLL_RX_BUF_SIZE];

static void UartUapiPollTask(void *arg)
{
    (void)arg;

    uart_pin_config_t pins = {
        .tx_pin = POLL_UART_TX_PIN,
        .rx_pin = POLL_UART_RX_PIN,
        .cts_pin = PIN_NONE,
        .rts_pin = PIN_NONE,
    };

    /* 与 sdk uart_demo 一致: 8 数据位 */
    uart_attr_t attr = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_BIT_8,
        .stop_bits = 1,
        .parity = 0,
        .flow_ctrl = 0,
    };

    uart_buffer_config_t buf_cfg = {
        .rx_buffer = s_rx_buf,
        .rx_buffer_size = POLL_RX_BUF_SIZE,
    };

    if (uapi_uart_init(POLL_UART_BUS, &pins, &attr, NULL, &buf_cfg) !=
        ERRCODE_SUCC) {
        printf("[uart_uapi_poll] uapi_uart_init failed\r\n");
        return;
    }
}

```

```

    }

    printf("[uart_uapi_poll] poll bus %d echo 115200 8N1 (TX=MGPIO8
RX=MGPIO7)\r\n", (int)POLL_UART_BUS);

    while (1) {
        uint32_t n = 0;
        while (n < POLL_RX_BUF_SIZE &&
!uapi_uart_rx_fifo_is_empty(POLL_UART_BUS)) {
            int32_t r = uapi_uart_read(POLL_UART_BUS, s_rx_buf + n, 1,
POLL_RW_TIMEOUT);
            if (r != 1) {
                break;
            }
            n++;
        }
        if (n > 0) {
            (void)uapi_uart_write(POLL_UART_BUS, s_rx_buf, n, POLL_RW_TIMEOUT);
        }
        IoTWatchDogKick();
        osDelay(MS_TO_TICKS(10));
    }
}

static void UartUapiPollEntry(void)
{
    osThreadAttr_t attr = {
        "UartUapiPollTask", 0, NULL, 0, NULL, POLL_TASK_STACK_SIZE,
POLL_TASK_PRIO, 0, 0
    };

    if (osThreadNew(UartUapiPollTask, NULL, &attr) == NULL) {
        printf("[uart_uapi_poll] Failed to create task\r\n");
    }
}

APP_FEATURE_INIT(UartUapiPollEntry);

```

BUILD.gn

```

# Copyright (c) 2026 SZTU Open Source Organization.
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

static_library("uart_uapi_poll_demo") {
    sources = [ "uart_uapi_poll.c" ]
}

```

```

defines = [
    "CONFIG_UART_SUPPORT_TX",
    "CONFIG_UART_SUPPORT_RX",
]

include_dirs = [
    "//commonlibrary/utils_lite/include",
    "//kernel/liteos_m/kal/cmsis",
    "//base/iothardware/peripheral/interfaces/inner_api",
    "//device/soc/hisilicon/ws63v100/sdk/include/driver",

    "//device/soc/hisilicon/ws63v100/sdk/drivers/chips/ws63/rom/drivers/chips/ws63/porting/pinctrl",
    "//device/soc/hisilicon/ws63v100/sdk/kernel/osal/include",
    "//device/soc/hisilicon/ws63v100/sdk/drivers/drivers/hal/uart",
    "//device/soc/hisilicon/ws63v100/sdk/drivers/chips/ws63/porting/uart",
    "//device/soc/hisilicon/ws63v100/sdk/middleware/utils/common_headers/native",
    "//device/soc/hisilicon/ws63v100/sdk/drivers/chips/ws63/include",
]
}

```

实现要点： `uapi_uart_read` 在空 FIFO 时易忙等，须先 `**uapi_uart_rx_fifo_is_empty`，再按小长度读取；循环内 `**osDelay` + 喂狗。

九、拓展实验二：UART 中断接收 + 任务回显

本节不提供完整参考答案。请在理解第八节轮询实验与 [09-uapi_uart驱动接口与使用说明.md](#) 的前提下，独立完成 `uart_uapi_irq` 工程。

9.1 实验目的

- 使用 `**uapi_uart_register_rx_callback` 注册接收回调，理解 **中断上下文** 与 **任务上下文** 的分工。
- 在回调中**只做短操作**（如拷贝数据）；在**任务**中调用 `**uapi_uart_write` 完成回显。
- 了解 WS63 下单次 RX 回调 `**length` 常见约 **≤16 字节**（见 [09-uapi_uart驱动接口与使用说明.md](#) 3.4），长数据可能分多次回调。

9.2 功能要求（必须满足）

- 新建工程目录 `**uart_uapi_irq`：执行 `**python3 applications/make_app.py -c uart_uapi_irq`（或 `**e`），得到 `uart_uapi_irq.c` 与 `BUILD.gn` 模板。
- 硬件与总线**：默认仍使用 `**UART_BUS_2` + `**S_MGPI08 / S_MGPI07`（与第八节一致）；若改用 UART0，须书面说明风险（见完整版第八节）。
- `**BUILD.gn`：须定义 `**CONFIG_UART_SUPPORT_TX`、`**CONFIG_UART_SUPPORT_RX`，并包含与 `**uart_uapi_poll` 相同级别的 `**uart.h` 及 HAL/porting `include_dirs`（可参考第八节 `uart_uapi_poll/BUILD.gn`，改 `**sources` 与静态库目标名为 `**uart_uapi_irq_demo`）。
- 初始化**：`uapi_uart_init(..., extra_attr=NULL, buf_cfg=有效 rx 缓冲)`，波特率等与第八节一致。

5. **注册回调**：调用 `**uapi_uart_register_rx_callback`，**条件建议使用**

`**UART_RX_CONDITION_FULL_OR_SUFFICIENT_DATA_OR_IDLE**`，`size` 可取 1**（含义以 `uart.h` 为准）。

6. **回调函数**：

- 参数含 `**buffer`、`length`、`error**`；若 `error` 或数据非法则返回。
- 将 `buffer` 中 `length` 字节拷贝到你自己的缓冲区或消息结构体（回调返回后 `buffer` 可能失效，禁止在回调里长期持有指针）。
- 使用 `**osMessageQueuePut**` 将数据投递给任务；在 LiteOS CMSIS 适配下，**中断里 Put 的 timeout 必须为 0**（参见 `kernel/liteos_m/kal/cmsis/cmsis_liteos2.c` 中逻辑）。

7. **任务线程**：创建消息队列；循环 `**osMessageQueueGet`（可带超时），**收到数据后**

`**uapi_uart_write` 回显；循环内 `**IoTWatchDogKick**`。

8. 入口使用 `**APP_FEATURE_INIT**` 创建上述任务（与第八节风格一致）。

9.3 注意

- 回调内：**不要** `printf`、**不要** 长循环、**不要** 在 ISR 里直接调用可能阻塞或长时间持锁的接口（如盲目调用 `**uapi_uart_write**` 可能与驱动锁冲突，**建议仅在任务里写**）。
- **不要** 与 `**uart_uapi_poll**` 同时链接进固件并初始化**同一路 UART**。
- 若编译报 `uart_uapi_irq_demo` 未加入 `config.py` / `ohos.cmake`，在教师指导下对照仓库或完整版配置补全（学生提交前须保证**可编译烧录**）。

9.4 验收标准

- 烧录后，**调试串口**能看到你打印的**一行启动提示**（内容自定，须含学号或姓名缩写之一以便批改）。
- 在 **MGPIO8/MGPIO7** 外接串口（或你书面说明的等效接线）上，**发送任意字节可回显**；无明显卡死、看门狗反复复位。
- 代码结构清晰：回调与任务职责分离；关键步骤有简要注释。

9.5 提交物

- 源码：`uart_uapi_irq.c`、`BUILD.gn`，以及你修改过的 `app/BUILD.gn` 中 `**features**` 片段说明（可截图或文字）。
 - 实验报告：说明中断与轮询的差异、回调里为何要用消息队列、遇到的问题与解决方法。
 - 串口回显截图 1 张。
-